

Aprendizaje Automático

Aprendizaje supervisado

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada

Contenido

- 1 Aprendizaje automático
- 2 Aprendizaje supervisado
- 3 Ejemplo: Clasificación binaria
- 4 Evaluación y rendimiento de los modelos
- 5 Validación del modelo
- 6 Ejercicios y entrega

Contenido

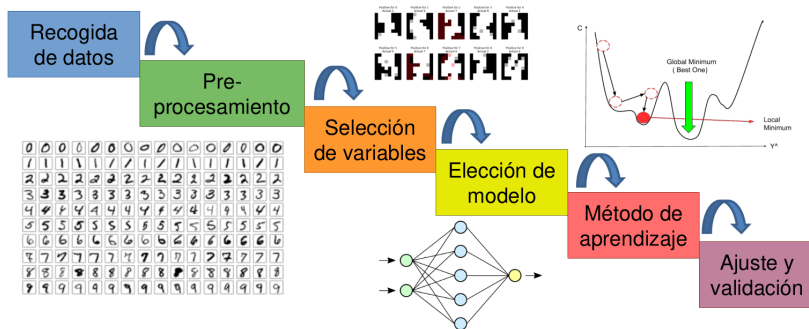
- 1 Aprendizaje automático
- 2 Aprendizaje supervisado
- 3 Ejemplo: Clasificación binaria
- 4 Evaluación y rendimiento de los modelos
- 5 Validación del modelo
- 6 Ejercicios y entrega

Hay 3 tipos básicos de aprendizaje automático:

- **Supervisado**: Dado un conjunto de patrones (datos) de entrada X , se desea predecir el valor de otro patrón de salida y . El conjunto de datos (X, y) se conoce a priori.
- **No Supervisado**: Existe un conjunto de patrones (datos) de entrada X , pero **no existen patrones de salida** y .
- **Por Refuerzo**: No existen datos X ni y a priori. Se aprende a partir de la experiencia.

Aprendizaje Automático

El diseño de una solución con aprendizaje automático pasa por varias fases:



Aprendizaje Automático: Terminología

A partir de ahora usaremos los siguientes términos:

- **Modelo**: Mecanismo formal implementable el cual, para un valor de entrada de un problema dado, proporcionará como salida su solución. Los modelos suelen estar **parametrizados**; es decir, su comportamiento depende de un conjunto de valores (normalmente los notaremos como $w = (w_1, w_2, \dots)$).
- **Algoritmo de aprendizaje**: Método automático utilizado para que el modelo proporcione las salidas deseadas para las entradas proporcionadas. Normalmente consiste en encontrar los valores de parámetros w que hacen que el rendimiento del modelo sea óptimo.
- **Función de pérdida**: Notada normalmente como $\mathcal{L}(y, w)$. Se trata de una función a optimizar (minimizar) **por el algoritmo de aprendizaje** para que el modelo funcione correctamente.
- **Métricas de rendimiento**: Utilizadas para **evaluar** el buen o mal rendimiento del modelo a la hora de resolver el problema.

Contenido

- 1 Aprendizaje automático
- 2 Aprendizaje supervisado**
- 3 Ejemplo: Clasificación binaria
- 4 Evaluación y rendimiento de los modelos
- 5 Validación del modelo
- 6 Ejercicios y entrega

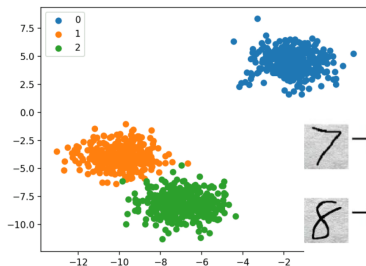
En el Aprendizaje Supervisado, se asume conocido un conjunto de datos E/S $\{(X_i, y_i)\}_{i=1}^L$, con valores de entrada $X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,n})$.

Problemas típicos del aprendizaje supervisado :

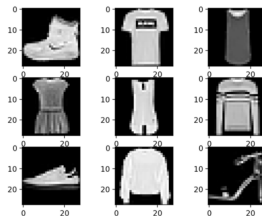
- **Clasificación:** Se desea conocer a qué categoría o categorías (**clases**) $y_i \in C$ puede pertenecer un patrón de entrada X_i .
- **Regresión/aproximación funcional:** Dado $y_i \in \mathbb{R}$ un valor numérico, se espera poder calcular una aproximación $\hat{y}_i = f(X_i)$ tal que $\hat{y}_i \approx y_i$.

Aprendizaje Supervisado

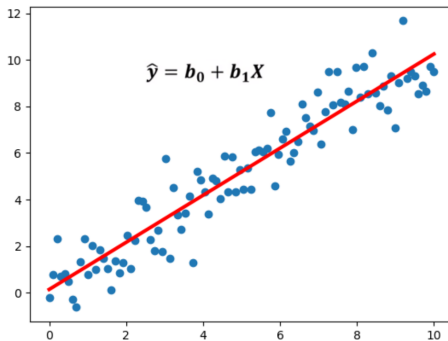
Ejemplos de problemas de clasificación:



7 → 7 5 → 5
8 → 8 3 → 3
2 → 2 4 → 4



Ejemplos de problemas de regresión:

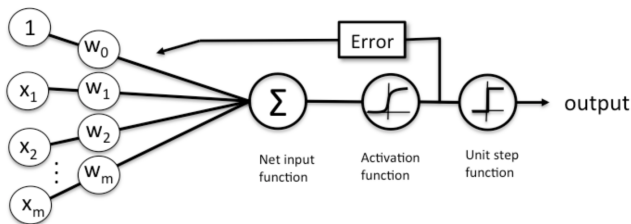


Contenido

- 1 Aprendizaje automático
- 2 Aprendizaje supervisado
- 3 Ejemplo: Clasificación binaria**
- 4 Evaluación y rendimiento de los modelos
- 5 Validación del modelo
- 6 Ejercicios y entrega

Clasificación binaria: Regresión logística

Ejemplo de **modelo** para clasificación binaria: **regresión logística**.



Schematic of a logistic regression classifier.

Figura 1: Obtenida desde

https://rasbt.github.io/mlxtend/user_guide/classifier/LogisticRegression/

El modelo de **regresión logística**, aplicado sobre un problema de clasificación binaria, necesita un total de $n + 1$ parámetros $w = (w_0, w_1, w_2, \dots, w_n)$.

La salida del modelo se corresponde con la probabilidad de predecir $p(y = 1|x, w)$

Se calcula como:

$$S = w_0 + \sum_{i=1}^n x_i w_i$$

$$p(y = 1|x, w) = f(S) = \frac{1}{1 + e^{-S}}$$

Ejemplo de implementación rápida:

```
import numpy as np

# Función para calcular la probabilidad de salida de un modelo de regresión logística
def LogisticRegressionProbability(x, w, debug= True):
    S= w[0] + x @ w[1:].reshape(-1, 1)
    if debug:
        print('Calculo S para todos los X de entrada: \n{}'.format(S))

    logistic_function= 1/(1+np.exp(-S))
    if debug:
        print('Tras aplicar f(S): \n{}'.format(logistic_function))

    return logistic_function.squeeze()

# Función para calcular la clase (0,1) a predecir, dada una probabilidad p de entrada
def predict(p):
    return (p>=0.5).astype(float).squeeze()
```

Regresión logística: Algoritmo de Aprendizaje

OBJETIVO: Encontrar los parámetros w que hacen que el modelo resuelva correctamente el problema.

w se desconoce a priori. No podemos dar unos valores iniciales ya conocidos que funcionen (si pudiésemos, entonces no necesitaríamos aprender).

```
dataset= np.array([[0, 0, 0], # AND(0,0)
                  [0, 1, 0], # AND(0,1)
                  [1, 0, 0], # AND(1,0)
                  [1, 1, 1] # AND(1,1)
                  ], dtype=float)
# Cogemos entradas y salidas
X= dataset[:, :-1]
y= dataset[:, -1]

# Seleccionamos parámetros
w= np.array([-1.5, 1, 1], dtype=float)

# predecimos salidas del dataset
y_pred= predict( LogisticRegressionProbability(X, w) )

for y_real, y_output in zip(y, y_pred):
    print('El modelo devuelve {} y debería devolver {}'.format(y_output, y_real))
```

Regresión logística: Algoritmo de Aprendizaje

El **algoritmo de aprendizaje** se encarga de encontrar el modelo que mejor resuelve el problema.

Si el modelo está parametrizado con parámetros w , se trata de encontrar los parámetros w óptimos del mismo.

Normalmente se plantea como un problema de **optimización** para minimizar una función del error que comete el modelo.

En aprendizaje automático es habitual encontrar modelos parametrizados, y utilizar **funciones de error derivables** que permitan identificar la diferencia entre la salida esperada y_i y la salida $\hat{y}_i = f(x^i)$ del modelo para un patrón de E/S (x^i, y_i) .

Regresión logística: Algoritmo de Aprendizaje

El **método del descenso del gradiente** es un **algoritmo de aprendizaje** básico que trata de minimizar la función de error de forma iterativa.

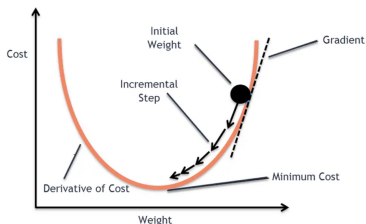
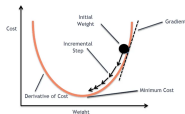


Figura 2: <https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>

En cada iteración, se modifican los parámetros w según las siguientes ecuaciones.

$$w = w + \Delta w \quad ; \quad \Delta w = -\lambda \frac{\partial}{\partial w} \mathcal{L}(y, w)$$

Regresión logística: Algoritmo de Aprendizaje



En las ecuaciones:

$$w = w + \Delta w \quad ; \quad \Delta w = -\lambda \frac{\partial}{\partial w} \mathcal{L}(y, w)$$

- Δw indica **cuánto** y **en qué dirección** debemos modificar el parámetro w
- λ es un **hiperparámetro** para seleccionar la proporción del *salto*/cambio que **queremos** hacer para modificar w (normalmente valor pequeño).
- $\mathcal{L}$ es una **función de error/coste/pérdida** del modelo (a minimizar).

Regresión logística: Algoritmo de Aprendizaje

Implementación en función **fit**:

```
def fit(X, y, model= LogisticRegressionProbability, w_ini= None,
        loss_function= BCE, derivative_loss_function= diff_BCE_LogReg,
        lambda= 0.01, n_itera= 10, debug=True):

    loss_history= [] # Guardar la evolución de loss function

    # Copiamos parametros iniciales
    w= w_ini.copy() if w_ini is not None else np.random.rand(X.shape[1]+1)

    # Iteramos un número de veces dado
    for i in range(n_itera):

        # Cálculo de p
        p= model(X, w, debug= False)

        # Cálculo de función de pérdida
        loss= loss_function(y, p)
        loss_history.append(loss)

        # ACTUALIZACION DE PARAMETROS
        dw= derivative_loss_function(X, y, p, w)
        w-= lambda*dw

        if debug:
            print('Iteración {}. loss={}'.format(i+1, loss))

    # Salida final
    p= model(X, w, debug= False)
    loss= loss_function(y, p)
    loss_history.append(loss)
    if debug:
        print('FIN. loss={}'.format(loss))
    return w, loss_history
```

Regresión logística: Función de pérdida

La **función de pérdida** es una **medida de error o de rendimiento** que permite guiar el aprendizaje (es decir, la **optimización** de los parámetros w del modelo).

Por ejemplo, la función **Binary Cross Entropy (BCE)** es útil para guiar el aprendizaje en problemas de clasificación binaria:

$$\mathcal{L}(y, p) = -\frac{1}{L} \sum_{j=1}^L (y_j \log(p_j) + (1 - y_j) \log(1 - p_j))$$

- L : Número de patrones existente en el conjunto de datos.
- y_j : Salida esperada para el j -ésimo patrón.
- p_j : Probabilidad proporcionada por el modelo para el j -ésimo patrón.

Regresión logística: Función de pérdida

Implementación en función **BCE**:

```
# Implementación de la función de pérdida BCE
def BCE(y, p):

    # Evitamos probabilidades de 0
    p[p==0] = 0.000001

    # Cálculo
    bce = -np.sum( y*np.log(p) + (1-y)*np.log(1-p) )
    return bce
```

Regresión logística: Función de pérdida y aprendizaje

La adaptación de la **función de pérdida BCE** al algoritmo del **Descenso del Gradiente** para el **modelo de Regresión Logística** implica manejar las siguientes fórmulas:

$$\begin{aligned} S &= w_0 + \sum_{i=1}^n x_i w_i \\ p &= \frac{1}{1+e^{-S}} \\ \mathcal{L}(y, w) &= -\frac{1}{L} \sum_{j=1}^L (y_j \log(p_j) + (1 - y_j) \log(1 - p_j)) \end{aligned}$$

Usamos la regla de la cadena para descomponer la derivada de la función de pérdida:

$$\frac{\partial}{\partial w_i} \mathcal{L}(y, w_i) = \frac{\partial \mathcal{L}(y, w_i)}{\partial p_i} \frac{\partial p_i}{\partial S_i} \frac{\partial S_i}{\partial w_i}$$

... y así podemos tratar cada término por separado.

Regresión logística: Función de pérdida y aprendizaje

Para $\mathcal{L}(y_j, p_j) = -(y_j \log(p_j) + (1 - y_j) \log(1 - p_j))$, calculamos:

$$\frac{\partial \mathcal{L}(y_j, p_j)}{\partial p_j} = \frac{p_j - y_j}{p_j(1 - p_j)}$$

Para $p_j = \frac{1}{1 + e^{-S_j}}$, calculamos:

$$\frac{\partial p_j}{\partial S_j} = p_j(1 - p_j)$$

Para $S_j = w_0 + \sum_{i=1}^n x_i^j w_i$, calculamos:

$$\frac{\partial S_j}{\partial w_i} = \begin{cases} 1 & i = 0 \\ x_i^j & i > 0 \end{cases}$$

Regresión logística: Función de pérdida y aprendizaje

Al integrar todos los términos para calcular

$$\frac{\partial}{\partial w_i} \mathcal{L}(y, w_i) = \frac{\partial \mathcal{L}(y, w_i)}{\partial p_i} \frac{\partial p_i}{\partial S_i} \frac{\partial S_i}{\partial w_i}$$

obtenemos:

$$\frac{\partial}{\partial w_i} \mathcal{L}(y, w_i) = \frac{1}{L} \sum_{j=1}^L \left(\frac{p_j - y_j}{p_j(1 - p_j)} p_j(1 - p_j) x_i^j \right)$$

$$\frac{\partial}{\partial w_i} \mathcal{L}(y, w_i) = \frac{1}{L} \sum_{j=1}^L \left((p_j - y_j) x_i^j \right)$$

(Para simplificar la fórmula estamos asumiendo un valor $x_0^j = 1$)

Regresión logística: Función de pérdida y aprendizaje

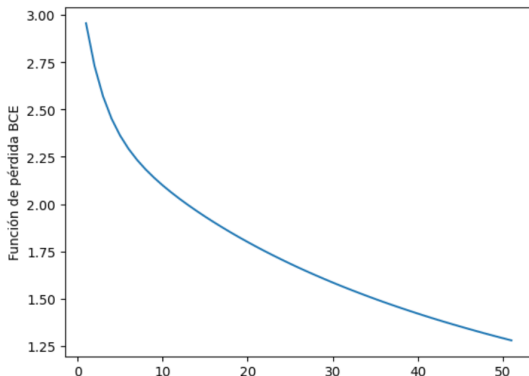
Implementación en función `diff_BCE_LogReg`:

```
def diff_BCE_LogReg(X, y, p, w):  
    diff_outputs= (p-y).reshape(-1, 1)  
  
    # Extendemos X con 1 al principio  
    X_extended= np.hstack( (np.ones((len(X), 1)), X) )  
  
    # Cálculo de la derivada para cada patrón  
    dw_individual= diff_outputs*X_extended  
  
    # Agregación de todos los patrones en la función de pérdida  
    dw= np.mean(dw_individual, axis= 0)  
    return dw
```

Regresión logística: Ejemplo de uso

Ejemplo de uso para aprender a clasificar una función $AND(x_1, x_2)$:

Cargar el cuaderno **CustomLogisticRegression.ipynb** en vuestro GDrive.
Abridlo con Colab.



Contenido

- 1 Aprendizaje automático
- 2 Aprendizaje supervisado
- 3 Ejemplo: Clasificación binaria
- 4 Evaluación y rendimiento de los modelos**
- 5 Validación del modelo
- 6 Ejercicios y entrega

Evaluación y rendimiento: Métricas

Las **métricas** se utilizan para evaluar el rendimiento de un modelo. **Normalmente son distintas a la función de pérdida.**

Por ejemplo: Entrenar un modelo de regresión logística para clasificación binaria con función de pérdida **BCE**, y evaluarlo según la **tasa de aciertos**.

Usamos la siguiente nomenclatura:

- **TP (True Positives)**. Elementos de la clase objetivo (1) que han sido bien clasificados.
- **FP (False Positives)**. Elementos de la clase alternativa (0) que han sido clasificados en la clase objetivo (1).
- **TN (True Negatives)**. Elementos de la clase alternativa (0) que han sido bien clasificados.
- **FN (False Negatives)**. Elementos de la clase objetivo (1) que han sido clasificados en la clase alternativa (0).

Evaluación y rendimiento: Métricas

Algunas **métricas** usadas en **clasificación**:

- **Accuracy**. Tasa de aciertos total, calculada como
$$Acc = \frac{TP+TN}{TP+TN+FP+FN}$$
- **Precision**. Tasa de aciertos entre los predichos como positivos, calculada como $Precision = \frac{TP}{TP+FP}$.
- **Recall**. Tasa de aciertos entre los reales positivos, calculada como $Recall = \frac{TP}{TP+FN}$.
- **F1-Score**. Métrica que agrupa la precisión y el recall. Calculada como:

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

Ver implementación en el cuaderno **CustomLogisticRegression.ipynb**.

Evaluación y rendimiento: Métricas

Algunas **métricas** usadas en **regresión**:

- **SE (Square Error)**. Error cuadrático, calculado como $SE = \sum_{i=1}^L (\hat{y}_i - y_i)^2$
- **MSE (Mean Square Error)**. Error cuadrático medio, calculado como $MSE = \frac{1}{L} \sum_{i=1}^L (\hat{y}_i - y_i)^2$
- **MAE (Mean Absolute Error)**. Error absoluto promedio, calculado como $MAE = \frac{1}{L} \sum_{i=1}^L |\hat{y}_i - y_i|$.
- **MAPE (Mean Absolute Percentage Error)**. Error porcentual absoluto promedio, calculado como $MAPE = \frac{1}{L} \sum_{i=1}^L \left| \frac{\hat{y}_i - y_i}{y_i} \right|$.

donde $\hat{y}$ es el valor predicho por el modelo e y el valor real esperado.

Evaluación y rendimiento: Métricas

Encontramos distintas métricas en **SkLearn** dentro del paquete **sklearn.metrics**.

- `accuracy_score`
- `auc`
- `confusion_matrix`
- `f1_score`
- `precision, recall`
- `mean_absolute_error`
- `mean_squared_error`
- **etc., etc., etc., ...**

Contenido

- 1 Aprendizaje automático
- 2 Aprendizaje supervisado
- 3 Ejemplo: Clasificación binaria
- 4 Evaluación y rendimiento de los modelos
- 5 Validación del modelo**
- 6 Ejercicios y entrega

Selección del modelo

La selección del modelo a usar depende de muchos factores:

- Necesidad de interpretabilidad/explicabilidad del comportamiento del mismo.
- Complejidad (tamaño) y número de parámetros.
- Idoneidad para el problema que se desea resolver.

Normalmente, es habitual probar distintos modelos para verificar cuál es el que proporciona un mejor rendimiento.

SkLearn tiene una amplia variedad de modelos para los tipos de aprendizaje supervisado y no supervisado.

Más información en https://scikit-learn.org/1.6/supervised_learning.html.

El problema del sobreajuste

El hecho de que obtengamos un valor “*bueno*” en una **métrica de evaluación** o en la **función de pérdida**, **sobre los mismos datos de entrenamiento**, no implica que el modelo sea el mejor posible.

Sobreajuste: resultados excepcionales en datos de entrenamiento en modelos complejos. **Poco generalizable a otras instancias del problema.**

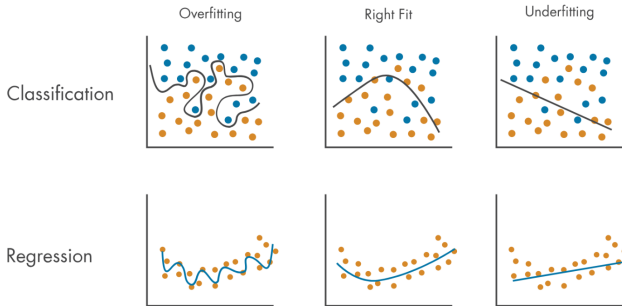
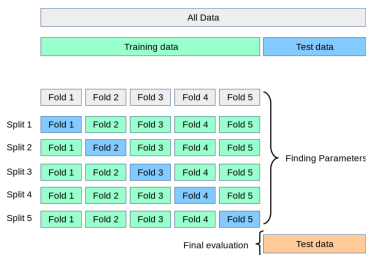


Figura 3: <https://la.mathworks.com/discovery/overfitting.html>

El problema del sobreajuste

El sobreajuste se puede evitar/paliar:

- Partiendo de modelos sencillos con bajo rendimiento e ir haciéndolos más complejos sucesivamente.
- Dividiendo el conjunto de datos en entrenamiento+test. **una de las claves principales es no usar los datos de test para absolutamente nada** (sólo verificar que los valores de las métricas son consistentes con los de entrenamiento).
- Usando técnicas de **validación** (por ejemplo, validación cruzada).



El problema del sobreajuste

Copia el fichero **SkLearnMetricsEvaluation.ipynb** a tu cuenta de GDrive y estudia cómo utilizar diferentes métricas y técnicas de evaluación, incluyendo:

- División en entrenamiento y test.
- Comprobación del rendimiento usando validación por cruce.

En el mismo cuaderno tienes ejemplos de modelos sobreajustados, ¡nunca construyas uno así!.

Contenido

- 1 Aprendizaje automático
- 2 Aprendizaje supervisado
- 3 Ejemplo: Clasificación binaria
- 4 Evaluación y rendimiento de los modelos
- 5 Validación del modelo
- 6 Ejercicios y entrega**

Ejercicios y entrega de la práctica

El cuaderno **P1.ipynb** contiene instrucciones para llevar a cabo la práctica 1 de la asignatura.

Se debe entregar únicamente el cuaderno por PRADO. **IMPORTANTE:**

- Prestad atención al análisis y descripción de lo que se hace.
- Cuidad el preprocesamiento de datos.
- Seleccionad varios modelos y “jugad” con sus hiperparámetros.
- **Discusión** sobre los resultados obtenidos por cada modelo por separado, y **conclusiones** finales indicando si hay alguno mejor que otro o no, **justificando y analizando** el porqué.
- No entregar el análisis o una justificación de lo que se hace es como no entregar nada.

PROBLEMA 1: Clasificación (5 puntos).

- Contexto: Mercado de telefonía móvil.
- Uso de Inteligencia Artificial para el desarrollo de un clasificador que indique la categoría o gama a la que debería pertenecer un terminal móvil.

PROBLEMA 2: Regresión (5 puntos).

- Contexto: Ingeniería Civil.
- Uso de Inteligencia Artificial para predecir la resistencia del hormigón.
- Se trata de modelar la relación entre la resistencia del hormigón (medida en MegaPascuales, MPa), con respecto a diferentes variables relativas a la mezcla.

Aprendizaje Automático

Aprendizaje supervisado

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada